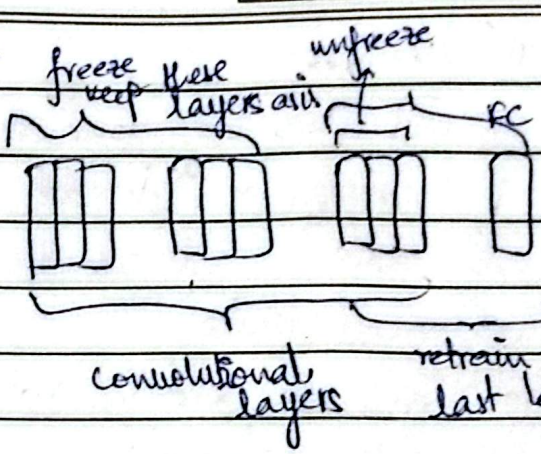
Date:

M T W T F S S

WAYS OF DOING TRANSFER LEARNING :-

- Feature Extraction
- Fine Tuning



for ex :- Image classif → cat/dog

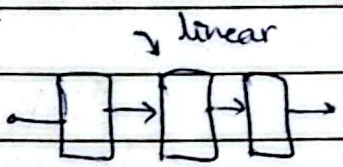
If model/architecture can already classify cat/dog now move on to identify features of cat/dog.

Problems with Sequential Model :-

ANN ↔ CNN

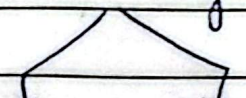
↳ Sequential - keras

- → 1 input
- → 1 output
- → linear



Examples where this Sequential Models won't work :-

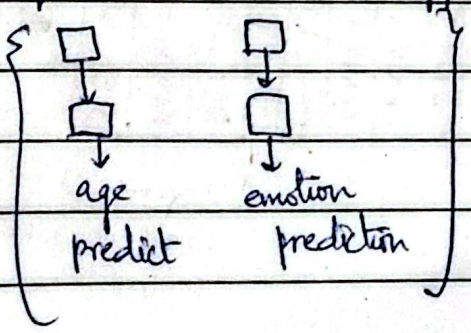
Example →  → image dataset → human faces → 25000 images

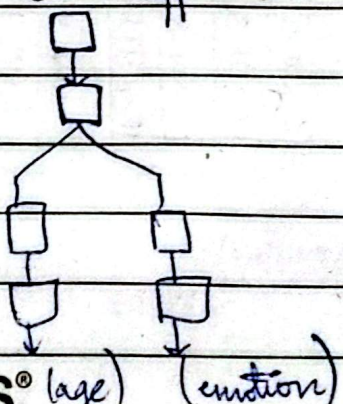


↔ Now we have to predict both age & emotion detection.

age : emotion : happy sad anger

sequential model → not appropriate {better approach}





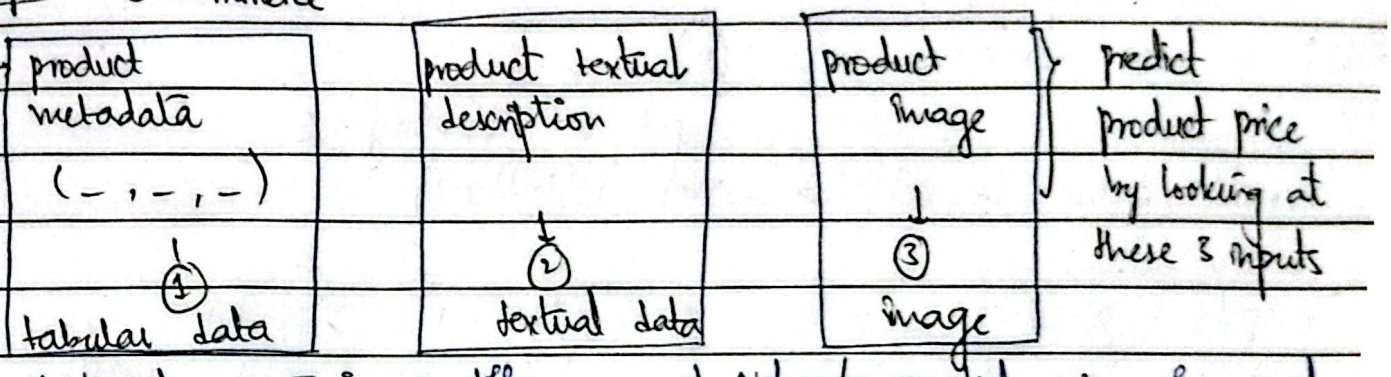
Page No.

Signature

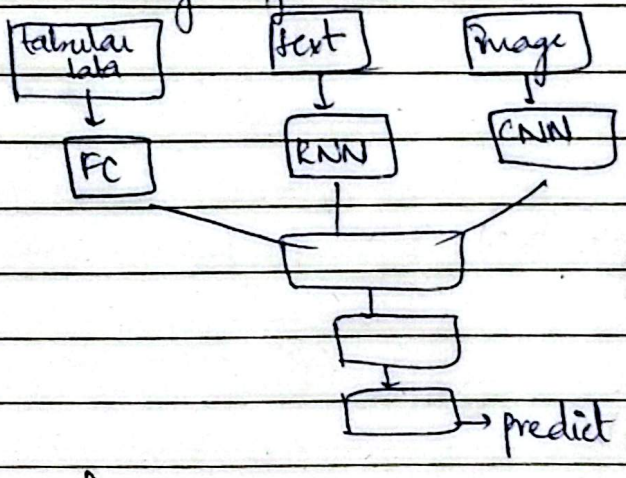
Date:

M	T	W	T	F	S	S

Example:- e-commerce

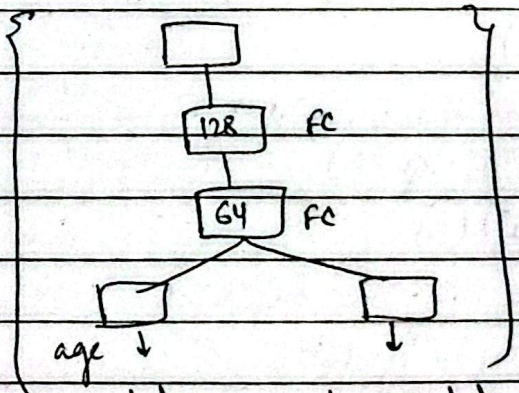


Naive Approach → Train 3 diff. Neural Networks, predict price from each & then take average of all that.



FUNCTIONAL API :-

Ex:- yearly salary | height | marital status — 3 cols → age, delhi/mumbai




```
from keras.models import Model
model = Model(inputs = x, outputs = [output1, output2])
from keras.layers import *
```

Date: M T W T F S S

```
x = Input(shape=(3,))
hidden1 = Dense(128, activation='relu')(x)
                                  ↑ Input to hidden layer 1
hidden2 = Dense(64, activation='relu')(hidden1)

output1 = Dense(1, activation='linear')(hidden2)   # predicts age
output2 = Dense(1, activation='sigmoid')(hidden2)  # predicts location
model.summary()

from keras.utils import plot_model
plot_model(model)
plot_model(model, show_shapes=True)
```

MULTI INPUT MODEL :-

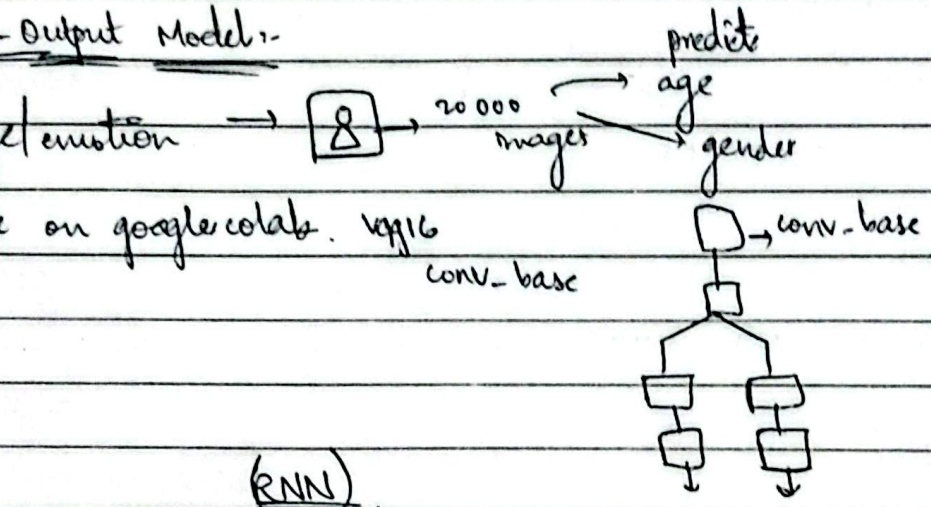
```
from keras.models import Model
from keras.layers import *
# define two sets of inputs
inputA = Input(shape=(32,))
inputB = Input(shape=(128,))
# the first branch operates on the first input
x = Dense(8, activation="relu")(inputA)
x1 = Dense(4, activation="relu")(x)
# the second branch operates on the first input
y = Dense(64, activation="relu")(inputB)
y1 = Dense(32, activation="relu")(y)
y2 = Dense(4, activation="relu")(y1)
```

Date:

M | T | W | T | F | S | S

# combine the output of the two branches
combined = concatenate ([x1, x2])
# apply a FC layer & then a regression prediction on the combined outputs
z = Dense (2, activation = "relu") (combined)
z = Dense (1, activation = "linear") (z)
# our model will accept the input of the two branches & then
# output a single value.
model = Model (inputs = [inputA, inputB], outputs = z)
from keras.utils import plot_model
plot_model (model)

Multi-Output Model :-



Code on google colab. vgg16

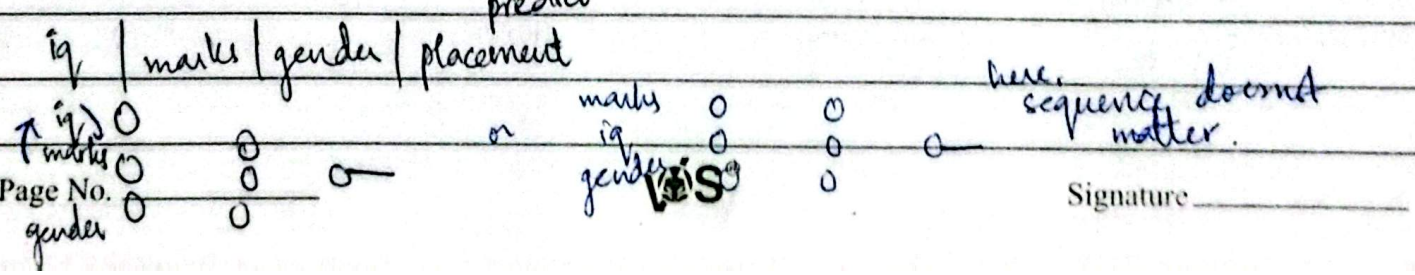
RNN

RECURRENT NEURAL NETWORK :-

Sequential Data

ANN → tabular data } CNN → images

RNN → type of sequential model designed to work on sequential data.



Page No.

Signature

Date: M T W T F S S

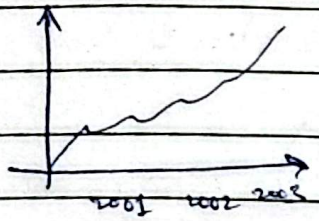
But, there are data where sequence matters

↳ eg → textual

Hi my name is kinz

↑

Time series (sequence matter) Kinz Hi my name is X (incorrect order)



↙ Sequential

trading (prices)

Speech :- ↙ Sequential

DNA Sequence :- ↙ Sequential

ACGTCAARAAC

RNN → NLP → ML

↗ ↳ RNN

RNN → why not ANN or CNN

→ RNN Why?

→ Application - RNN

Why use RNN?

ANN → text classification

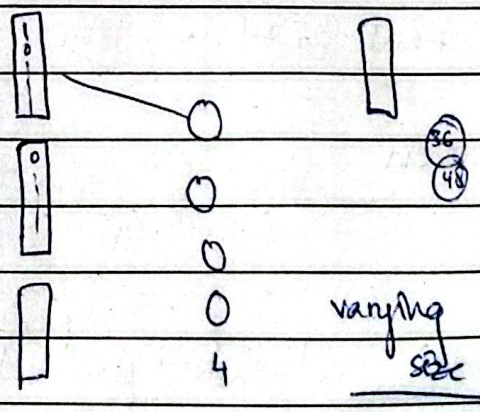
Input	output
Hi my name is kinz (5)	0
I love Campus X (3)	0
Dubai won the match (4)	1

12 items

vectorize

[1, 0, 0, 0, 0, ____]

Hi



Page No. _____ Signature _____

Date:

M T W T F S S

Problems of applying ANN on Sequential data.

→ size of textual data varies. (variable input size).

→ MLP / ANN cannot carry long term dependency (in case of long sentences).

→ Even if we use zero padding.

↳ alot of unnecessary computations.

→ Problem in predictions.

→ Biggest problem → we lose semantics (order of words since we are inputting all at the same time at the start).

→ Totally disregarding the sequence info. (no semantic meaning captured).

ANN ✗

RNN

sequence → [sequential data]

Applications of RNN.

→ Prediction of next word Autocompletion

→ sentiment analysis

→ google translate.

RNN THE WHAT?

(time steps, input-features)

Review	Sentiments
movie was good	1
movie was bad	0
movie was not good	0

vocab → [movie, was, not, good, bad]

one-hot encoding

movie [1 0 0 0 0]

was [0 1 0 0 0]

good [0 0 1 0 0]

bad [0 0 0 1 0]

not [0 0 0 0 1]

review 1 [[1 0 0 0 0], [0 1 0 0 0], [0 0 1 0 0]]

t_1 t_2 t_3

(3, 5)

no. of timesteps → # of features

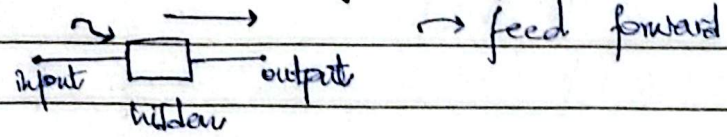
Date:

M	T	W	T	F	S	S

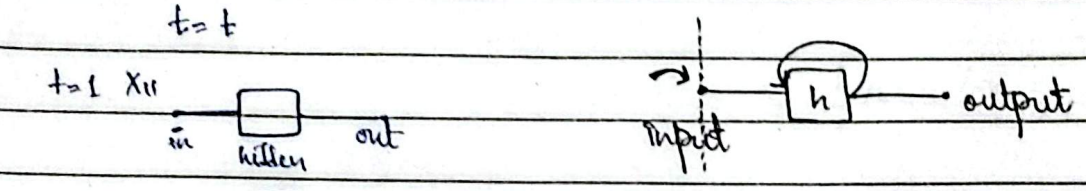
Keras → simple RNN → batch size, timesteps, input_features

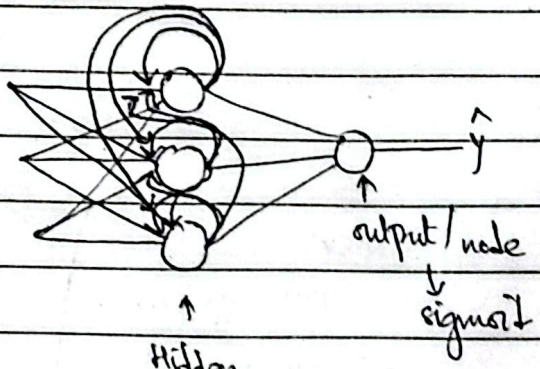
How RNN works?

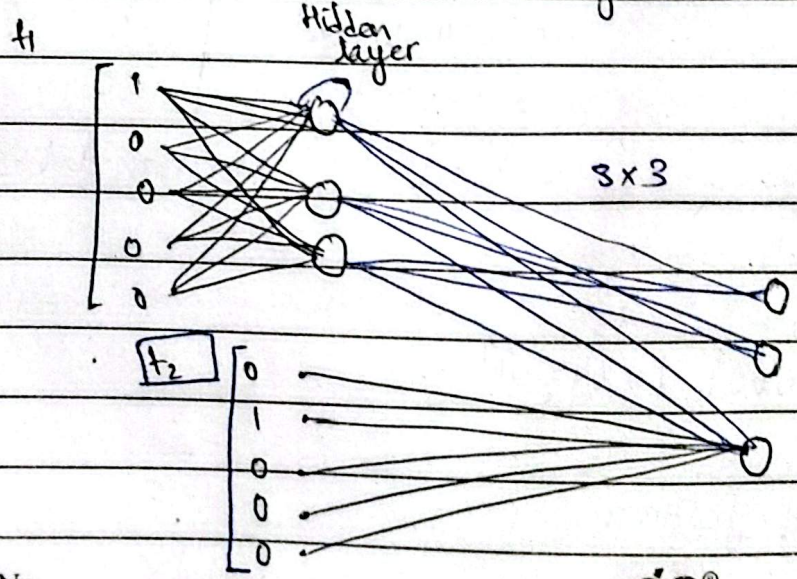
	Review	Sentiment
x_1	movie (x_{11}) was (x_{12}) good (x_{13})	1
x_2	movie (x_{21}) was (x_{22}) bad (x_{23})	0
x_3	movie (x_{31}) was (x_{32}) not (x_{33}) good (x_{34})	0



→ feed forward





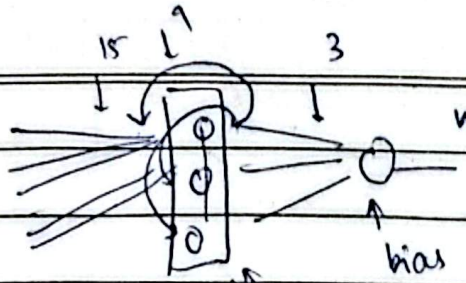


VOS®

Signature

Date:

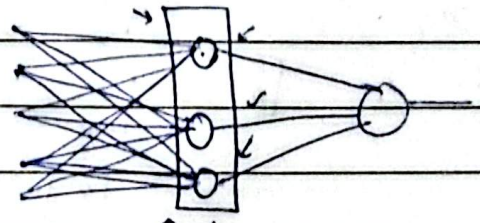
M	T	W	T	F	S	S



15 + 9 + 3 = 27 wts + 4 biases

```
from keras import Sequential
from keras.layers import Dense, SimpleRNN
model = Sequential()
model.add(SimpleRNN(3, input_shape=(4,5)))
model.add(Dense(1, activation="sigmoid"))
model.summary()
```

Review	sentiment
x_{11} x_{12} x_{13}	1
x_{21} x_{22} x_{23}	0
x_{31} x_{32} x_{33} x_{34}	0



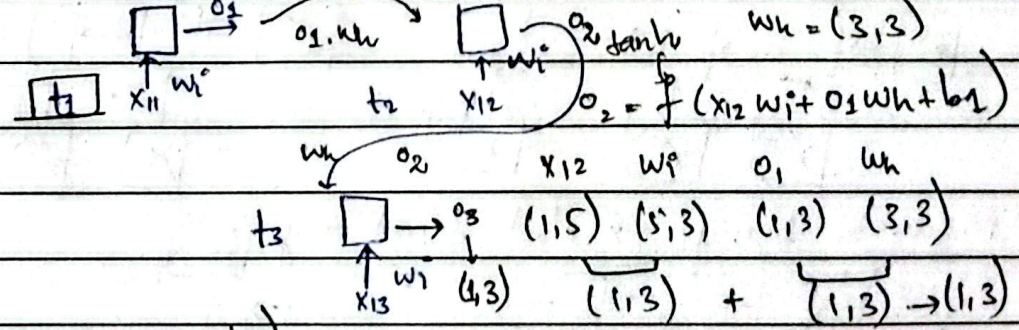
x_{11} → vectors 5 dim

[1 00 00]

one by one

relu/tanh loop

$x_{11} w_i \rightarrow (1,3)$ $\quad f(x_{11} w_i + b) = o_1 \rightarrow (1,3)$



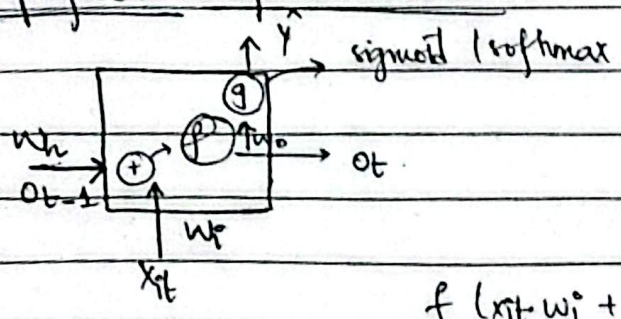
$o_2 = f(x_{12} \cdot w_i + o_1 \cdot w_h + b_1)$

$o_3 = f(x_{13} \cdot w_i + o_2 \cdot w_h + b_1)$

Date:

M	T	W	T	F	S	S

Simplified Representation :-



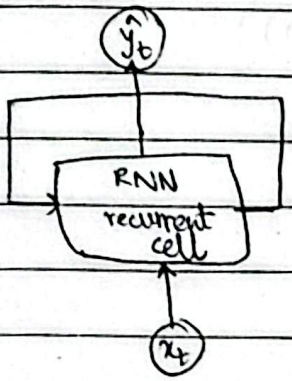
$\hat{y} = g(o_t W_o)$

$f(x_{it} W_i + o_{t-1} W_h) \rightarrow$ vector

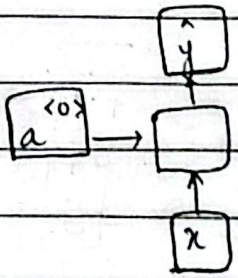
i = row of

t = time step

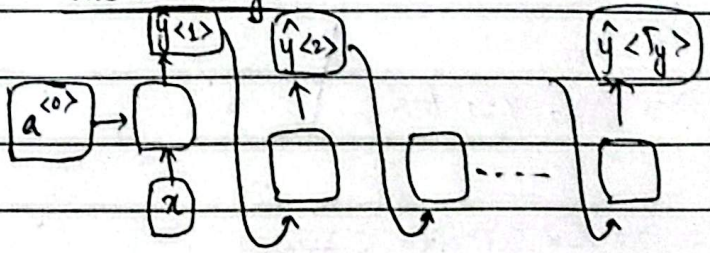
RNN.



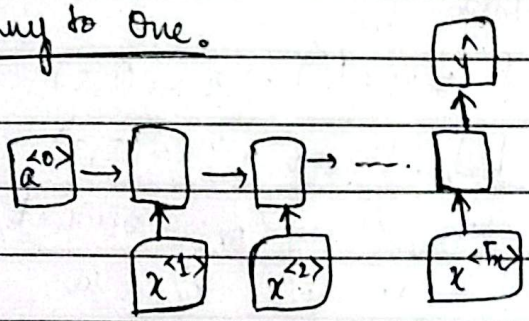
One to One.



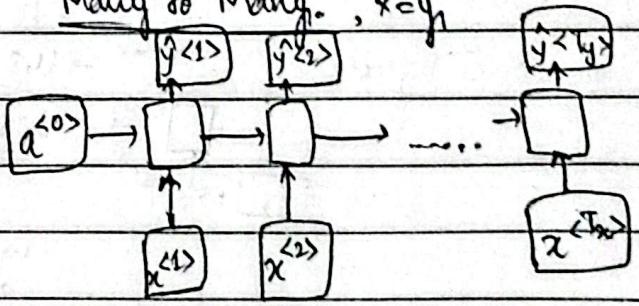
One to many.



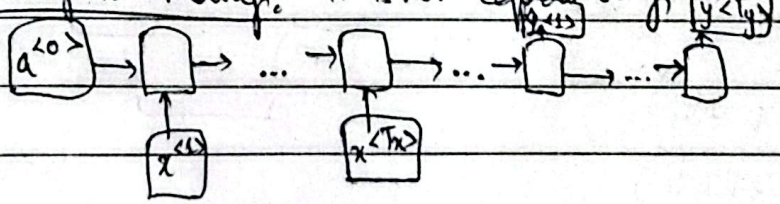
Many to One.



Many to Many. , x=y



Many to Many. x is not equal to y



Date:

M	T	W	T	F	S	S

prediction, hidden_state = rnn (word, prev_hidden_state)

prev_hidden_state = hidden_state

next_word_prediction = prediction

Computing Hidden state :-

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Hidden state h_t acts as a memory of the network. It summarizes all the information seen in the sequence upto time t. Each new input updates this memory.

Why we use tanh?

Because it's output is centered around zero & it helps stabilize training. However, it doesnot solve the vanishing Gradient problem. LSTM networks were introduced to address this issue.

Output Layer :- $\hat{y} = \underset{\text{sigmoid}}{\text{softmax}}(W_y h_t + b_y)$

1. Representing language to a Neural Network

words ⟶ numeric representation

Step 1 :- Generate vocabulary

Collect all unique words from your dataset.

Ex :- Dataset :- "I like to run", "I like to walk".

Vocabulary = ["I", "like", "to", "run", "walk"]

Step 2 :- Create Indexes.

Assign a unique number (index) to each word :-

"I" → 0

"like" → 1

"to" → 2

"run" → 3

"walk" → 4

Date: M T W T F S S

Step 3 :- One-Hot Encoding

- One-hot encoding represents each word as a sparse vector.
- Length of vector = total number of unique words.
- Only one element is 1, the rest are 0.
- Ex with vocabulary above :-

"I" → [1, 0, 0, 0, 0]

"walk" → [0, 0, 0, 0, 1]

Problem :- As vocabulary grows, vectors become huge & sparse, which is inefficient.

Step 4 :- Learned Embeddings

- Instead of sparse vectors, we map each word to a dense vector of lower dimensions (e.g. 50, 100 or 300).
- These vectors are learned during training.
- Similar words have vectors close to each other in space :-

Ex :- "run" & "walk" may have vector like [0.2, 0.9, -0.1] & [0.3, 0.85, -0.05]

Adv - lower dimensionality, - captures semantic similarity - can be used directly as input to RNNs, LSTMs, GRUs.

2. Design criteria for Language Models.

Handle variable-length sequences

- Sentences are not all the same length :-

"The food was great" → 4 words

"We visited a restaurant for lunch" → 6 words

Feed-Forward networks (FFN) require fixed-length input → cannot handle variable length naturally.

RNNs handle variable length because they process sequences word by word (time step by time step).

Model long-term dependencies

Words at the start of a sentence may influence words at the end

Date: M T W T F S S

- RNNs maintain a hidden state, updated at each step, which carries information from the past.

Capture sequence order

Words order matters :-

1. "Yesterday, I went for a walk".
2. "I went for a walk yesterday".

Even if the words are the same, the meaning may change. RNNs capture this because their hidden state depends on previous words.

Share parameters

The same weights are used at every time step of the sequence.

Why?

- Reduces number of parameters.
- Learns patterns once, regardless of position.
- Ex :- "I went for a walk" → learn the meaning of "went for a walk" once instead of at every position.

3. Why Padding is Needed in RNNs.

RNNs (& variants like LSTM, GRU) process sequences step by step, but training is usually done in batches for efficiency. Problem: sequences in a batch have different lengths.

Solution :- Padding

- Add special tokens (usually zeros) at the end of shorter sequences so all sequences in a batch have the same length.

Ex :- Sequences :-

[1, 2, 3] → pad to length 5 → [1, 2, 3, 0, 0]

[4, 5] → pad to length 5 → [4, 5, 0, 0, 0]

Why Padding is Needed.

- Uniform input size - Neural Networks process batches, so all inputs must

Date:
M T W T F S S

have the same shape.

- Faster computation :- Allows parallel processing using vectorization.
- Single computational graph :- No need to rebuild the model for different sequence lengths.

Bucketing (optimization)

- Group sequence of similar lengths (e.g. 10, 20, 50).
- Pad within each group instead of padding everything to maximum length.
- Reduces unnecessary padding & improves efficiency.

Quick summary :-

- Padding = same-length sequences
- Helps in batch training + efficiency
- Bucketing = smarter padding.

HOW BACKPROPAGATION WORKS IN RNN

→ RNN → Backprop → BPTT (Backpropagation through time).

Many to One RNN.

Sentiment Analysis

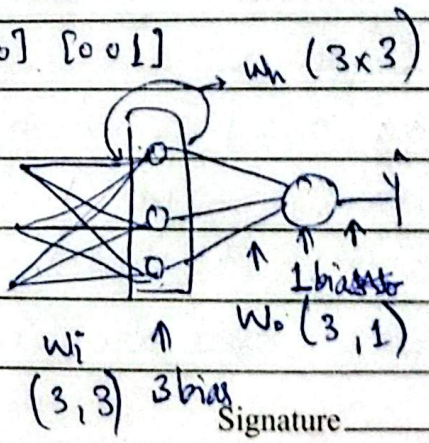
text → RTO → positive / negative

text			sentiment
cat	mat	rat	1
rat	rat	mat	1
mat	mat	cat	0
x_1	[1 0 0]	[0 1 0]	[0 0 1]	1
x_2	[0 0 1]	[0 0 1]	[0 1 0]	1
x_3	[0 1 0]	[0 1 0]	[1 0 0]	0

sentiments → vectors

vocab :-> cat mat rat

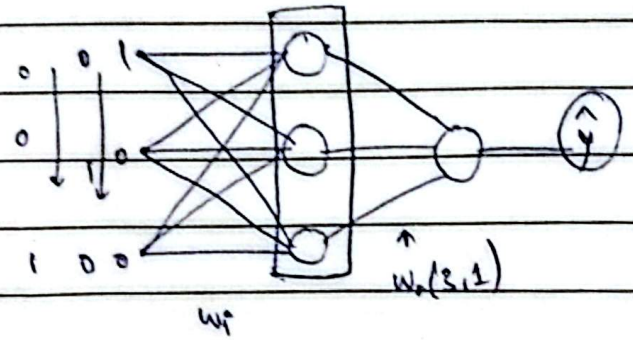
[1 0 0] [0 1 0] [0 0 1]



Date:

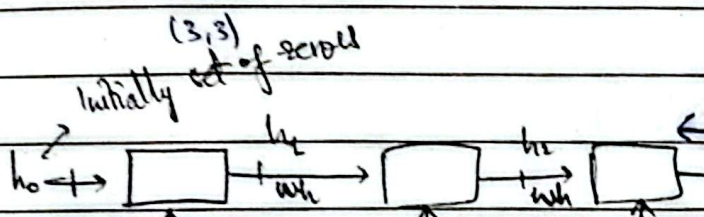
M	T	W	T	F	S	S

x_1 [1 0 0] [0 1 0] [0 0 1]



w_i (3,3) set of scroll

Initially



$L \to \hat{y} \to f(h_3 w_o + b)$

$\frac{\partial L}{\partial w_o} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_o}$

$\frac{\partial L}{\partial \hat{y}}$, $L = -y_i \log \hat{y}_i - (1-y_i) \log (1-\hat{y}_i)$

$h_1 = f(x_{11} w_i + h_0 w_h + b)$

$h_2 = f(x_{12} w_i + h_1 w_h + b)$

$h_3 = f(x_{13} w_i + h_2 w_h + b)$

$y \to \hat{y} = f(h_3 w_o + b)$

$L = -y_i \log \hat{y}_i - (1-y_i) \log (1-\hat{y}_i)$

loss calculate $\longrightarrow$ minimize

gradient descent

$w_i = w_i - \alpha \frac{\partial L}{\partial w_i}$, $w_h = w_h - \alpha \frac{\partial L}{\partial w_h}$, $w_b = w_b - \alpha \frac{\partial L}{\partial b}$ Backprop.

$\frac{\partial L}{\partial w_i}$, $L \to \hat{y} \to h_3 \to w_o$, $h_3 \to x_{13} w_i + h_2 w_h + b$

$\frac{\partial L}{\partial w_i} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial w_i}$

$\frac{\partial h_3}{\partial w_i} = x_{13}$ $\frac{\partial h_3}{w_h} = h_2$

$\frac{\partial \hat{y}}{\partial h_3} = w_o$ $\frac{\partial \hat{y}}{\partial h_2} = w_o$

$\frac{\partial L}{\partial w_h} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w_h}$

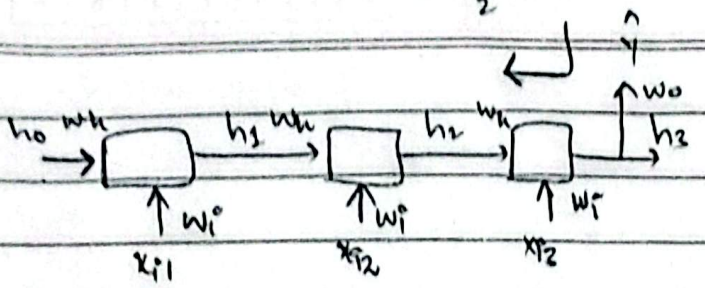
Page No.

Signature

L = Binary cross entropy loss or MSE

$L = \frac{1}{2}(\hat{y} - y)^2$

Date: M T W T F S S



$h_1 = f(h_0 w_h + x_{i1} w_i + b)$

$h_2 = f(h_1 w_h + x_{i2} w_i + b)$

$h_3 = f(h_2 w_h + x_{i3} w_i + b)$

$\hat{y} = f(h_3 w_0 + b)$

Loss depends on $\hat{y}$ (variable).

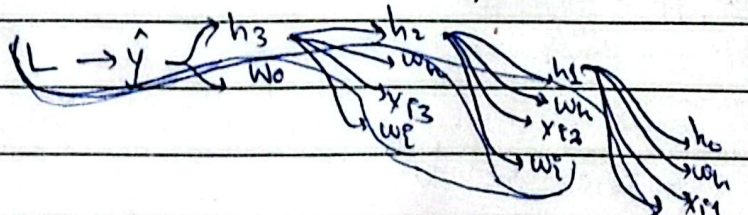
$L \rightarrow \hat{y} \leftarrow h_3, w_0$ $\quad \hat{y} = f(h_3 w_0 + b)$

$\frac{\partial \hat{y}}{\partial w_0} = h_3 \cdot f'(h_3 w_0 + b)$ ↓ derivative of $f(h_3 w_0 + b)$

$$\boxed{\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_0}}$$ ← If $L = \frac{1}{2}(\hat{y} - y)^2$

So, $\frac{\partial L}{\partial \hat{y}} = \frac{2}{2}(\hat{y} - y)^1 (1)$

$\frac{\partial L}{\partial w_i} \rightarrow (L \rightarrow \hat{y} \leftarrow h_3, w_0)$



$$\boxed{\frac{\partial L}{\partial w_i}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial w_i} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w_i} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_i}$$

3 terms. ↓ 3 words in each sentence that's why.

So, in short, we can write :-

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^{3} \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_j} \frac{\partial h_j}{\partial w_i}$$

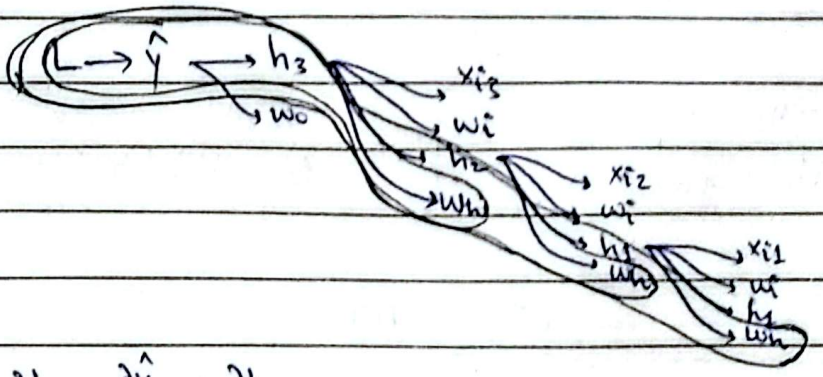
Date:

M	T	W	T	F	S	S

n = time steps

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^{n} \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_j} \cdot \frac{\partial h_j}{\partial w_i}$$

$$\frac{\partial L}{\partial w_h}$$



$$\frac{\partial L}{\partial w_h} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_2} \cdot \frac{\partial h_3}{\partial w_h} +$$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_h} +$$

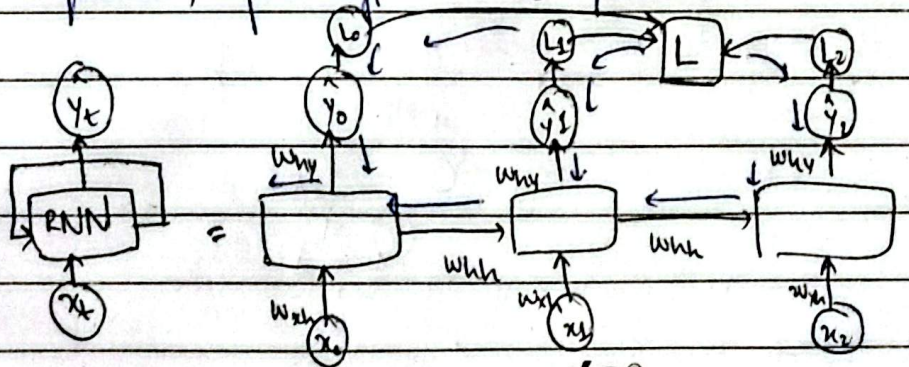
$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_h}$$

$$\boxed{\frac{\partial L}{\partial w_h} = \sum_{i=1}^{n} \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_j} \cdot \frac{\partial h_j}{\partial w_h}} \qquad w_h = w_h - \alpha \frac{\partial L}{\partial w_h}$$

n = timesteps

→ Training an RNN involves computing loss at each timestep & summing these losses to obtain the total loss.

→ The gradients are then propagated backward through the entire sequence, updating shared parameters used at every timestep.



Page No. _____

VIS®

Signature _____

Date:

M | T | W | T | F | S | S

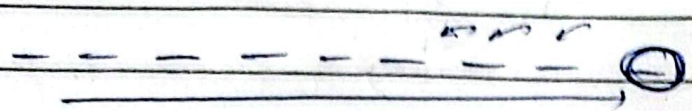
Problems With RNN.

RNNs → sequential data → textual, time series data

suffer (2) major problems → LSTMs (Why)

Problems with RNN

→ Problem of long term dependency. }
→ Problem of unstable gradients } → Unstable gradients, stagnated trainings.



# of timesteps

→ Cat sat on mat

short-term dependency.

→ Marathi is spoken in Mah

"short-term dependency"

→ Maharashtra is a beautiful place. I went there last year, but I couldn't enjoyed properly because I donot understand Marathi.

long term dependency.

↳ RNN doesnot perform well on this.

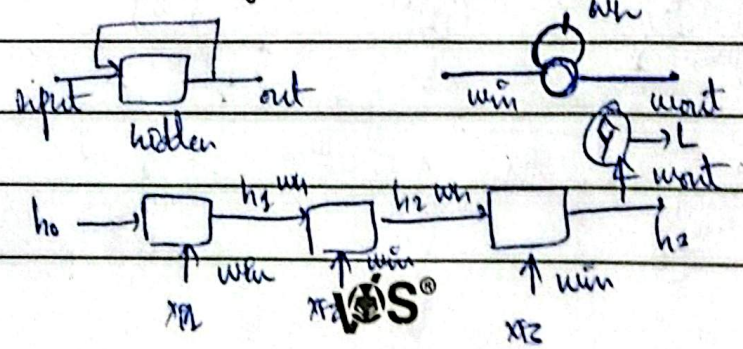
{ Vanishing gradient problem } →

{ Exploding gradient problem } → RNN face → possible sol.

Problem # 1 → Problem of long term dependency → Vanishing

input	output
→ 1 1 1	1
→ 0 0 → 1	0
0 0 0	0
1 1 1	1

3 time steps



Signature _____

Date:

M	T	W	T	F	S	S

(L) → min

w_h, w_{in}, w_{out}

↓

gradient des

$w_{in} = w_{in} - \alpha \frac{\partial L}{\partial w_{in}}$, $w_{out} = w_{out} - \alpha \frac{\partial L}{\partial w_{out}}$, $w_h = w_h - \alpha \frac{\partial L}{\partial w_h}$

$$\frac{\partial L}{\partial w_{in}} = \boxed{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_{in}}} + \boxed{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_{in}}} + \boxed{\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_{in}}}$$

↓ Short term dependency

↓ long term dependency.

This is only 3 time steps

Imagine for 100 time steps :-

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_{100}} \cdot \frac{\partial h_{100}}{\partial h_{99}} \cdot \cdots \cdots \cdots \cdots \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_{in}} \rightarrow \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_{100}} \prod \left(\frac{\partial h_t}{\partial h_{t-1}}\right) \frac{\partial h_1}{\partial w_{in}}$$

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_{100}} \prod_{t=2}^{100} \left(\frac{\partial h_t}{\partial h_{t-1}}\right) \frac{\partial h_1}{\partial w_{in}}$$

$h_1 = \tanh(x_1 w_{in} + h_0 w_h + b)$

$h_t = \tanh(x_t w_{in} + h_{t-1} w_h + b)$

$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(x_t w_{in} + h_{t-1} w_h + b) \cdot w_h$

$\tanh'(\,) \cdot w_h$; $\tanh'(\,)$: 0-1, w_h : value b/w 0-1.

≈ 0

whole term ≈ 0 close to 0.

This is Vanishing Gradient Problem.

Sol:-

1) Diff. activation fn. → relu / leaky relu

2) Better weight initialization

3) Better architecture → skip RNNs.

4) LSTM

Date:

M	T	W	T	F	S	S

Problem # 2. :- Exploding Gradient Problem :-

Assume we use relu instead of tanh

gradient becomes very very large.

or either your learning rate (α) very large

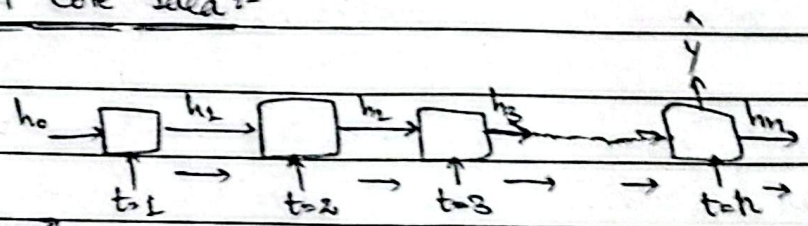
↓ Unstable Training (Exploding gradients)

Sol. :-

1) Gradient clipping
2) Controlled learning rate
3) LSTMs

LONG SHORT TERM MEMORY LSTM

LSTM Core Idea :-

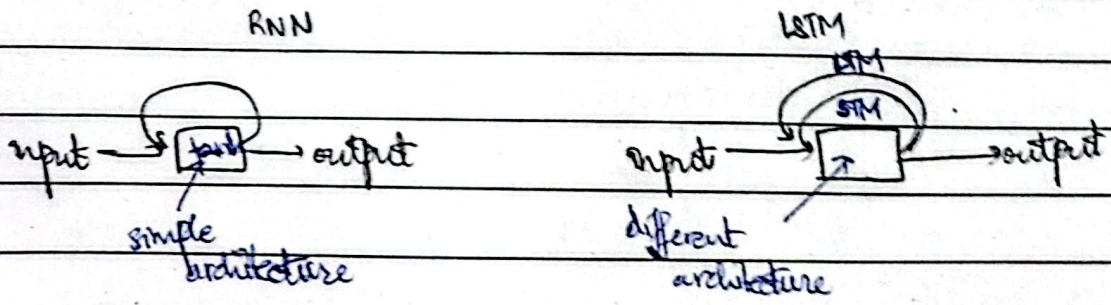


memory only this path to retain both short term context & long term context

2 paths

↳ one for STM

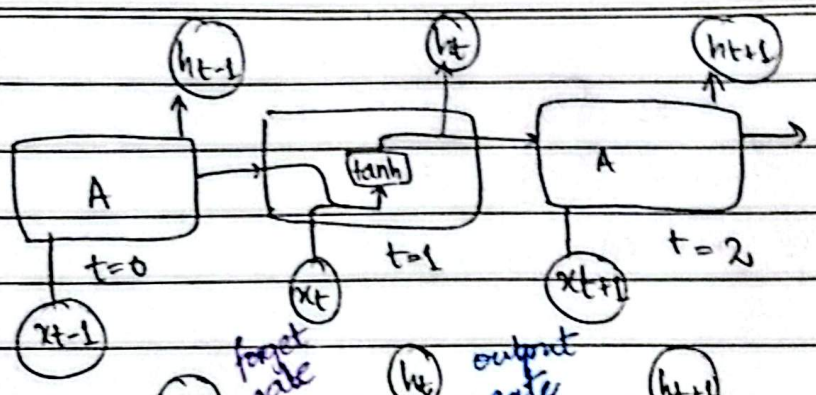
↳ one for LTM



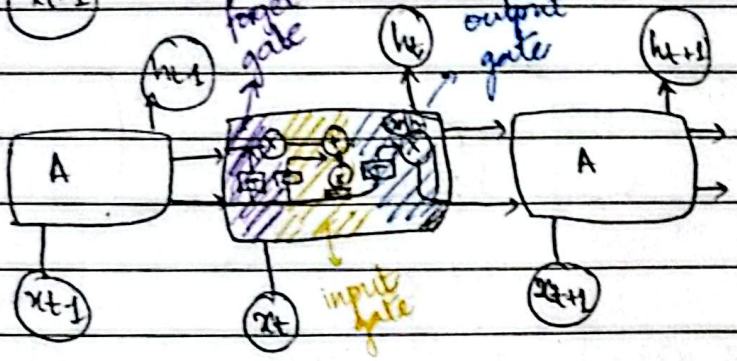
Date:

M	T	W	T	F	S	S

RNN.



LSTM.



→ two paths (one for retaining short term mem. other one for retaining long term mem.)

forget gate.

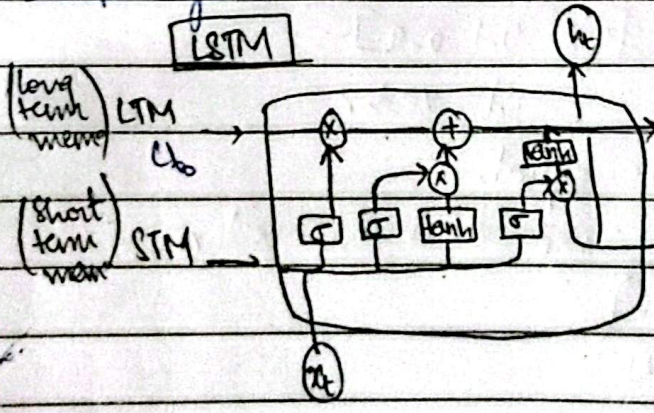
input wali se decides kisko LSTM long term se remove krna hai.

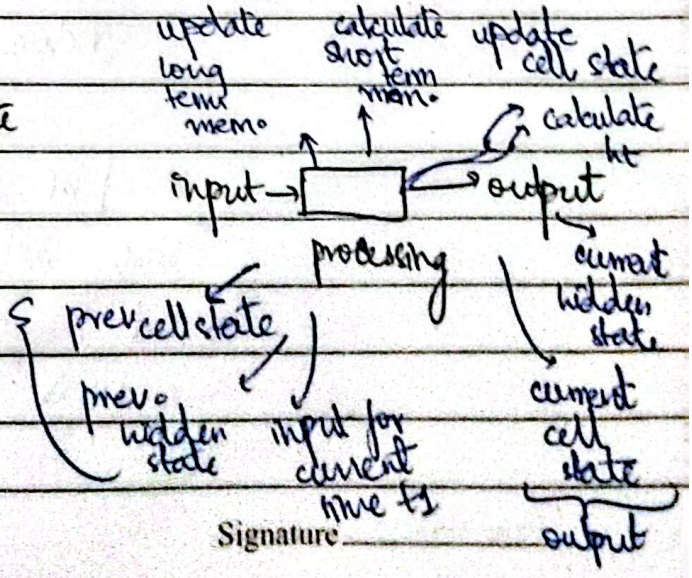
LTM → remove

input gate. what info. will be added in LTM.

LTM → add.

output gate. what will be the output.





Page No. _____ VIS® Signature _____

Date:

M	T	W	T	F	S	S

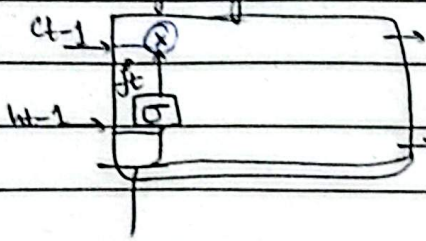
$c_{to} \rightarrow c_t$

update cell state → x_t → c_{to} remove

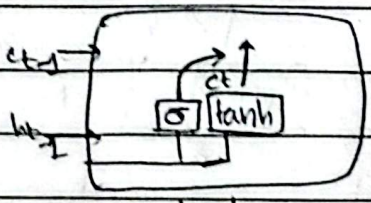
x_t → ☐ add

processing → 3 steps involved.

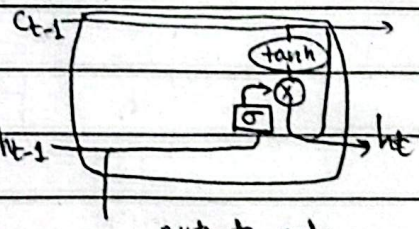
forget gate



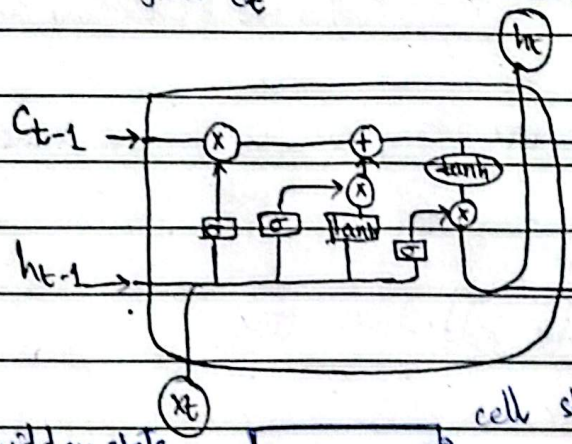
x_t to remove something from c_t



input gate to add something to the c_t.



x_t output gate calculate hidden state h_t.



hidden state short term mem. ← h_t c_t → cell state long term mem.

h_t c_t are like vectors → [0.5 0.1 0.9]

3d vector.

h_t c_t dim equal

both will have same dimension vectors

h_t [0.1 0.45 0.6]

c_t [0.55 0.6 0.0]

Date:

M	T	W	T	F	S	S

Sentiment Analysis Problem

x_t → vector

Input			vectorization	One Hot Encoding		
cat	mat	sat	0	cat	mat	sat
cat	sat	sat	0	1	0	0
mat	mat	cat	1	0	1	0
cat	mat		sat	0	0	1

→ [100] [010] [001]

$t=1$ $t=2$ $t=3$

[100] [010]

x_t x_t

x_t → word → vector

x_t is just a word converted to vector.

What are f_t, i_t, o_t & $\bar{c}_t$ → candidate cell state

f_t — forget gate

i_t $\bar{c}_t$ — input gate

o_t — output gate

c_t h_t

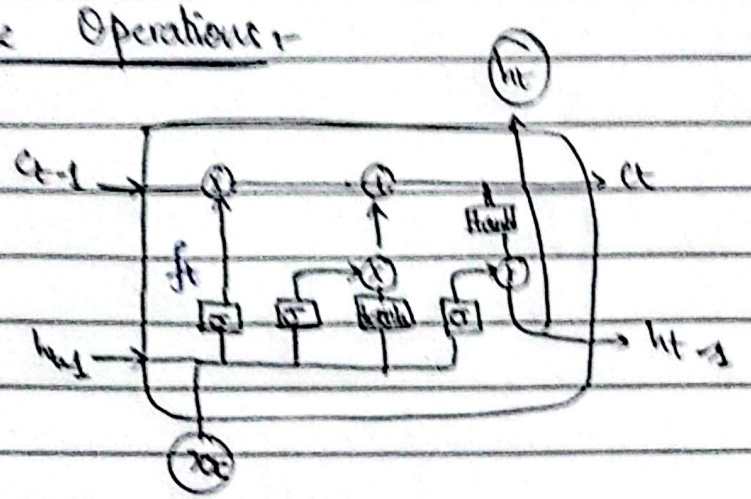
f_t i_t $\bar{c}_t$ o_t

→ all vectors will have same number of dimensions.

Date:

M	T	W	T	F	S	S

Point wise Operations :-



→ ⊙ Assume tanh(4) tanh(5) tanh(6)

→ ⊕ $C_{t-1} = [4\ 5\ 6] \rightarrow [0.20\ 0.34\ 0.53]$

→ tanh $f_t = [1\ 2\ 3]$

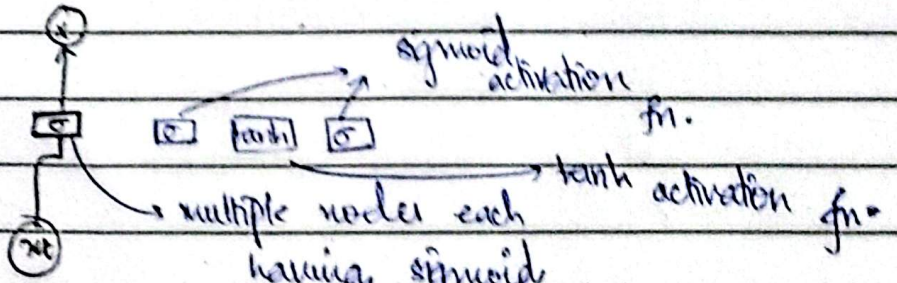
$C_{t-1} \otimes f_t \rightarrow$ vector

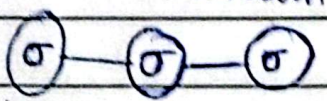
1×3 ← dimensions

Pointwise multiplication → [4 10 18]

Point wise addition → [5 7 9]

Neural Network Layers :-





num units / layers

→ hyperparameter → you decide it.

Date: _____

M	T	W	T	F	S	S

The forget gate :-

gets input

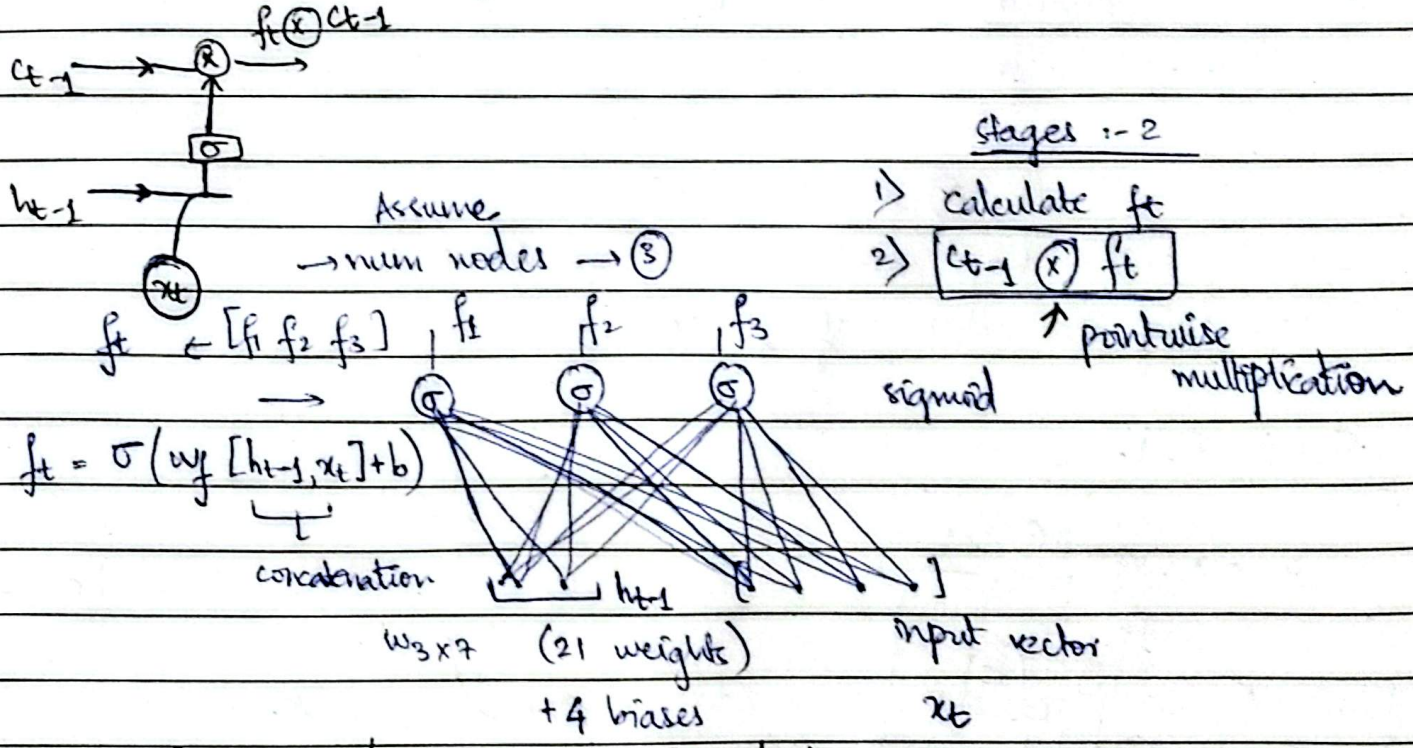
h_{t-1} prev. hidden state

c_{t-1} prev. cell state

x_t input vector

outputs :-

cell state → remove (remove what are not important for long term state).



$$f_t = \sigma(w_f [h_{t-1}, x_t] + b_f)$$

(3×1) (3×7) (7×1) (3×1)

shape. $3\times 1 + 3\times 1$

$f_t \otimes c_{t-1}$ ↓ removal

(c_{t-1}) → remove.

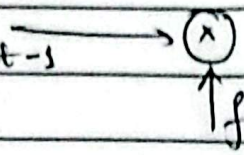
Signature _____

Date:

M T W T F S S

→ Why & how $f_t \otimes c_{t-1}$ is removing unnecessary things from long term memory (prev. cell state c_{t-1}).

forget gate



Assume

c_{t-1} is [4, 5, 6]

f_t = [½ ½ ½]

↓

$c_{t-1} \otimes f_t$ [2 2.5 3]

Assume $f_t \rightarrow$ [1 1 1]

c_{t-1} [4, 5, 6]

$c_{t-1} \otimes f_t$ [4 5 6]

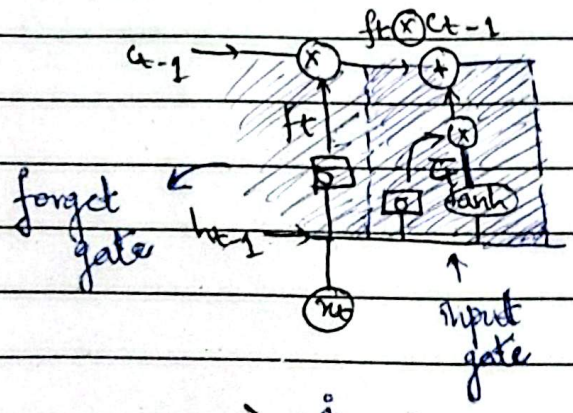
Assume $f_t \rightarrow$ [0 0 0]

$c_{t-1} \rightarrow$ [4 5 6]

$c_{t-1} \otimes f_t$ [0 0 0]

→ f_t decides how much info to go forward depending on value of f_t vector.

Input Gate :-



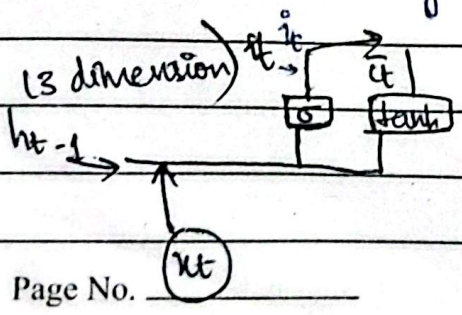
Stages :-

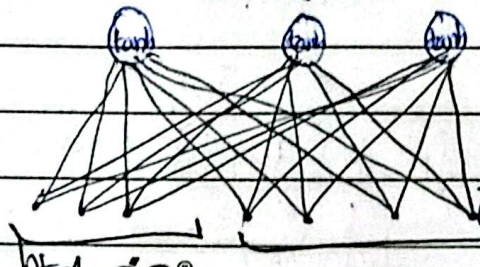
1) $\bar{c}_t$ calculate candidate self state

2) i_t calculate

3) Calculate c_t → current cell state

tanh Activation fn.





$= W_c$ (3x7)

$\bar{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$

Page No. _____ VIS® Signature _____

potential important information

Date:

M	T	W	T	F	S	S

$\bar{C}_t$ candidate cell state

$$C_t = \tanh\left(W_c\,[h_{t-1}, x_t] + b_c\right)$$

(3×7) (7×1) 3×1 3×1 3×1

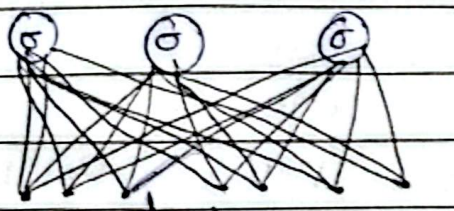
i_t → filter

$\bar{C}_t \rightarrow C_t$

Assumed 3 neurons

neurons (3)

sigmoid activation



W_i

h_{t-1} x_t

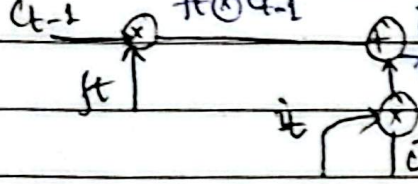
$$i_t = \sigma\left(W_i\,[h_{t-1}, x_t] + b_i\right)$$

(3×7) (7×1) (3×1) (3×1) (3×1)

addition

pointwise

$i_t \otimes \bar{C}_t = \bar{C}_f$ (filtered candidate cell state).



$$C_t = f_t \otimes C_{t-1} \oplus i_t \otimes \bar{C}_t$$

$t = t-1$ $[4\ 5\ 6]$ → (0% information loss) → $t = t$ $[4\ 5\ 6]$

$f_t = [1\ 1\ 1]$

$f_t \otimes \bar{C}_t = [0\ 0\ 0]$

$[4\ 5\ 6]$ $[0\ 0\ 0]$

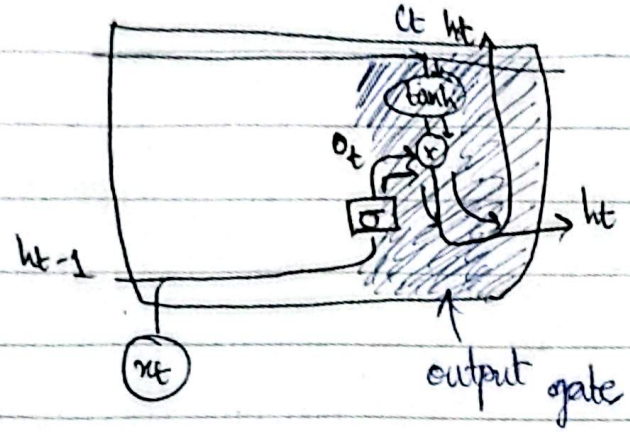
$C_t = [4\ 5\ 6]$

RNN drawback was it was not able to propagate long term context, but bcoz of this gate structure LSTM can do this.

DLP

LSTM

The Output Gate :-



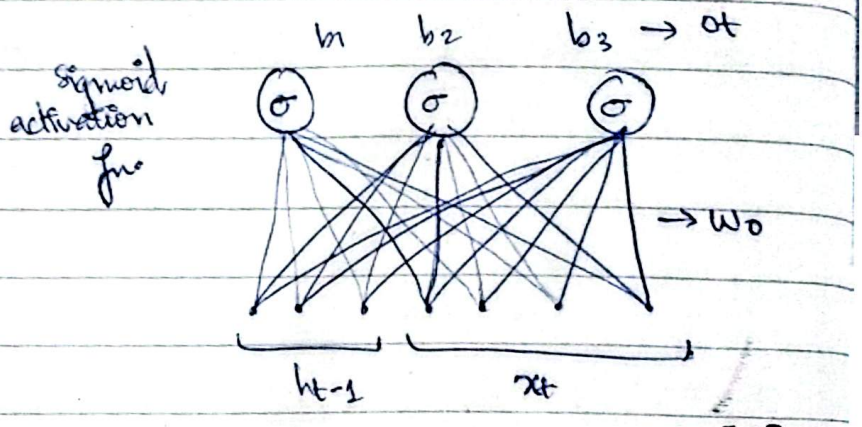
tanh(ct)

filtration

→ ot

$$h_t = [o_t \otimes \tanh(c_t)]$$

tanh(ct)	ot



→ W_o [ ]3x7

$$o_t = \sigma\left(W_o [h_{t-1}, x_t] + b_o\right)$$

o_t : (3x1)

W_o : (3x7), $[h_{t-1}, x_t]$: (7x1), b_o : 3x1

3x1 + 3x1

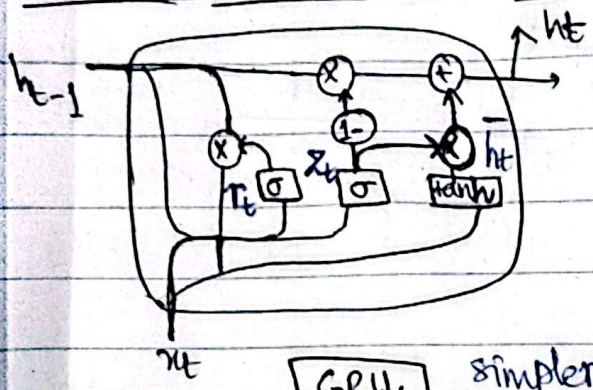
σ (3x1) → 3x1

$h_t = o_t \otimes \tanh(c_t)$

3x1 3x1 → 3x1

output for current time step.

Gated Recurrent Unit (GRU) :-



GRU simpler

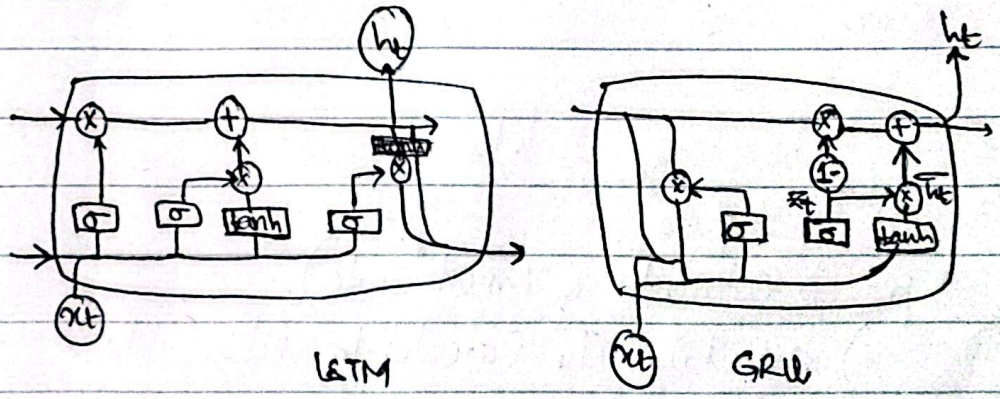
2 gates

→ In LSTM more parameters

complex

training time increases

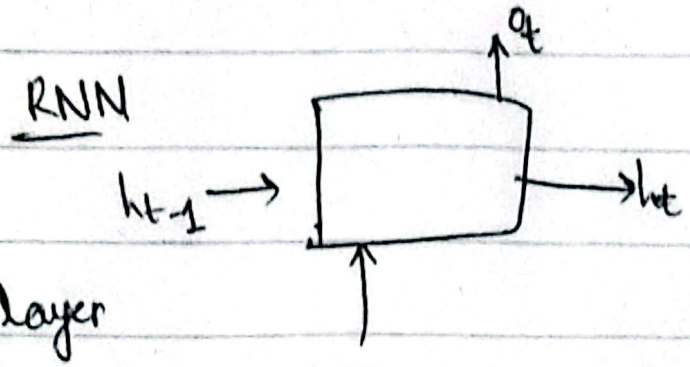
GRU ≃ LSTM



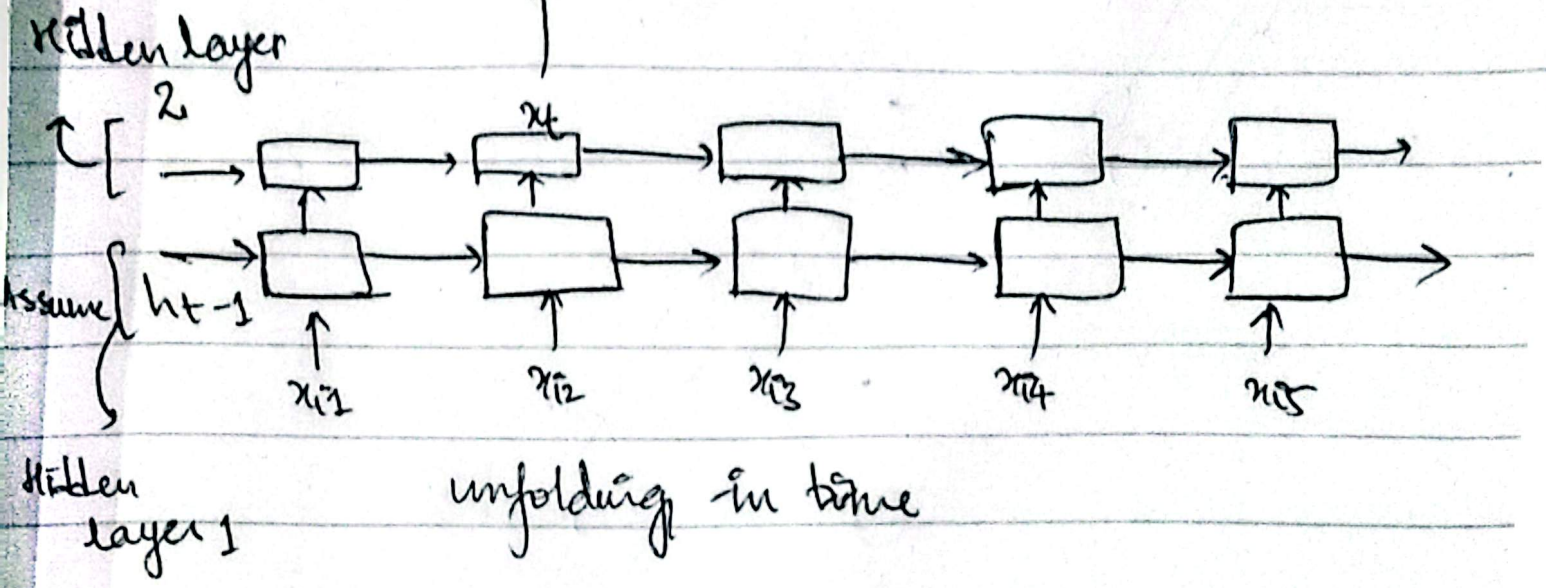
DEEP RNNs | STACKED RNNs

ANN

sentiment analysis



review	+ve / -ve

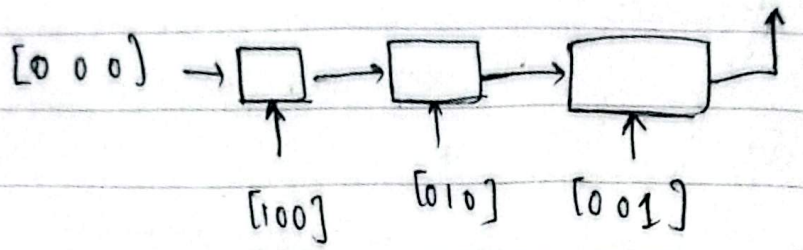


unfolding in time

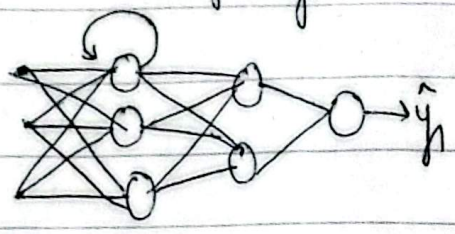
like this, we can stack layers of RNN.

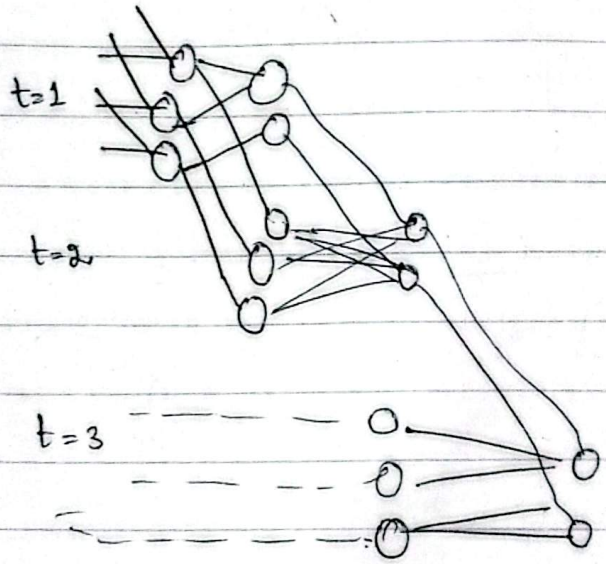
review	sentiment
cat mat rat	1 (positive)
rat rat mat	0 (negative)
mat cat cat	1 (negative)

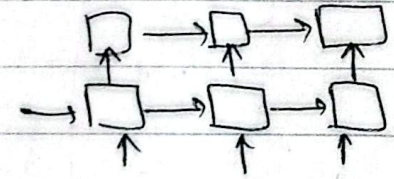
100 010 001



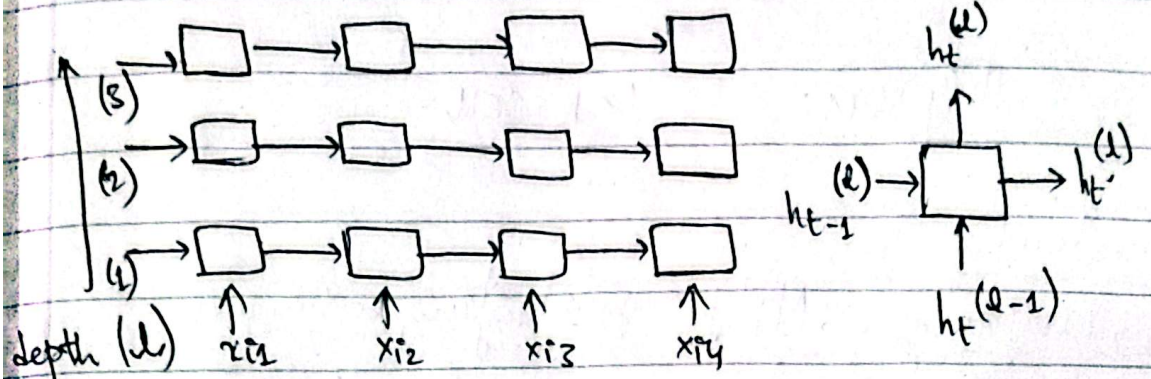
unfolding







Notation:-



$$\left[h_t^{(l)} = \tanh\left(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b \right) \right]$$

Why & When to use?

1. Hierarchical representation
2. Customization for Advanced Tasks

When to use Deep RNNs?

- complex tasks { speech recognition, machine translation }
- large datasets, overfitting
- if you have sufficient computational power
- when results not satisfactory of simpler models.

Variants :-

Simple RNN

- Deep LSTM
- Deep GRU

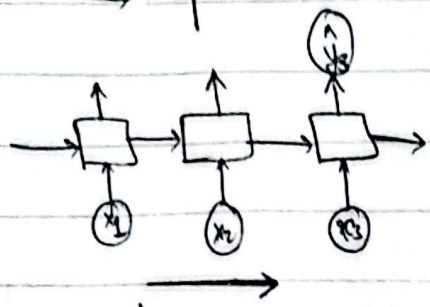
Disadvantages :-

↳ Overfitting → Apply regularization techniques, dropout etc.

↳ training time increases.

Bidirectional RNN :-

→ Deep RNN → BiRNN



NER → Named Entity Recognition

Ex :-

Sentence I love amazon. It's a greats website

I love amazon, It's a beautiful river.

river ?

website ? Amazon is river or website, this Entity Recognition depends on input ahead.

This is why we need BiRNN, we traverse input from both sides.

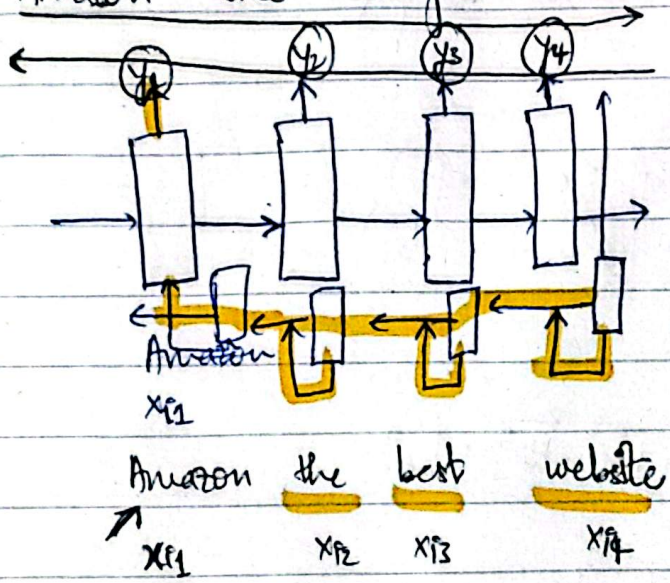
L ⟶ R left to right

L ⟵ R Right to left

$\overrightarrow{\text{RNN}}$ $\overleftarrow{\text{RNN}}$

Amazon the best website.

Amazon the beautiful river.



$$\overrightarrow{h_t} = \tanh\left(\overrightarrow{W}\overrightarrow{h_{t-1}} + \overrightarrow{U}x_t + \overrightarrow{b}\right)$$

$$\overleftarrow{h_t} = \tanh\left(\overleftarrow{W}\overleftarrow{h_{t+1}} + \overleftarrow{U}x_t + \overleftarrow{b}\right)$$

$$y_t = \sigma\left(V\left[\overrightarrow{h_t}, \overleftarrow{h_t}\right] + b\right)$$

BiRNN

Notation :-

Input sequence :-

$$x_1, x_2, \ldots, x_T$$

Forward hidden state :-

$$\overrightarrow{h_t}$$

Backward hidden state :-

$$\overleftarrow{h_t}$$

Forward RNN (left → right)

$$\overrightarrow{h_t} = f(W_x \cdot x_t + W_h \cdot \overrightarrow{h_{t-1}} + b)$$

Backward RNN (Right ← left)

$$\overleftarrow{h_t} = f(W'x_t + W'_h \cdot \overleftarrow{h_{t+1}} + b')$$

Final output :-

$$y_t = g(W_o[\overrightarrow{h_t}; \overleftarrow{h_t}] + b_o)$$

$f \rightarrow$ activation function

$g \rightarrow$ output activation function

$[;] \rightarrow$ concatenation of forward & backward states

Applications & Drawbacks :-

- NER (Named Entity Recognition)
- POS tagging
- Machine Translations
- Sentiment Analysis